

''''''

Operational Intelligence Automation Pipeline for Case Resolution Efficiency

Keywords: *Python, API, Data Engineering, Machine Learning, Pandas, NumPy, Scikit-Learn, Automation, ETL, Cloud, AsyncIO, Logging, Testing, Optimization*

Author: Maria Sultana

Date: 2025-11-11

Description:

This sample demonstrates an end-to-end Python workflow: fetching data from an external API, performing ETL (Extract, Transform, Load), running a predictive ML model, logging, and scheduling automation.

''''''

```
# =====  
  
# Imports: Core, Data, ML, Async  
  
# =====  
  
import asyncio  
  
import logging  
  
import requests  
  
import pandas as pd  
  
import numpy as np  
  
from datetime import datetime  
  
from sklearn.preprocessing import StandardScaler  
  
from sklearn.ensemble import RandomForestClassifier  
  
from sklearn.model_selection import train_test_split
```

```

from sklearn.metrics import classification_report

import joblib # For saving ML models


# =====

# Logging Configuration

# =====

logging.basicConfig(

    filename='advanced_pipeline.log',

    level=logging.INFO,

    format='%(asctime)s - %(levelname)s - %(message)s'

)

logging.info("Python data pipeline initialized.")


# =====

# Async API Fetching Function

# =====

async def fetch_api_data(url: str) -> pd.DataFrame:

    """

    Fetch data from a REST API asynchronously and convert it into a Pandas DataFrame.

    """

    logging.info(f"Fetching data from API: {url}")

    loop = asyncio.get_event_loop()

    response = await loop.run_in_executor(None, requests.get, url)

    if response.status_code == 200:

```

```

    data = response.json()

    logging.info(f"API data fetched successfully: {len(data)} records")

    df = pd.DataFrame(data)

    return df

else:

    logging.error(f"API fetch failed: {response.status_code}")

    return pd.DataFrame() # Return empty DataFrame on failure


# =====

# Data Cleaning & Transformation

# =====

def clean_transform_data(df: pd.DataFrame) -> pd.DataFrame:
    """
    Perform advanced data cleaning and feature engineering.
    """
    logging.info("Starting data cleaning and feature engineering.")

    # Drop missing values and duplicates

    df = df.dropna().drop_duplicates()

    # Create new features

    if 'timestamp' in df.columns:

        df['timestamp'] = pd.to_datetime(df['timestamp'])

        df['day_of_week'] = df['timestamp'].dt.dayofweek

        df['hour_of_day'] = df['timestamp'].dt.hour

```

```

# Encode categorical features

categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:

    df[col] = df[col].astype('category').cat.codes


logging.info(f"Data cleaned and transformed: {df.shape[0]} records, {df.shape[1]} features")

return df


# =====

# Machine Learning Pipeline

# =====

def run_ml_model(df: pd.DataFrame, target_col: str):

    """

    Train a RandomForestClassifier with robust metrics and save the model for deployment.

    """

    logging.info("Starting ML model training pipeline.")


    X = df.drop(columns=[target_col])

    y = df[target_col]


    # Feature scaling

    scaler = StandardScaler()

    X_scaled = scaler.fit_transform(X)


    # Train-test split

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

# Train Random Forest Classifier
clf = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)
clf.fit(X_train, y_train)

# Predictions and evaluation
y_pred = clf.predict(X_test)
report = classification_report(y_test, y_pred)
logging.info(f"ML Model Classification Report:\n{report}")

# Save model and scaler for production deployment
joblib.dump(clf, 'random_forest_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
logging.info("ML model and scaler saved for deployment.")

# =====

# Main Execution: Async + Pipeline
# =====

async def main():
    # Example API URL (replace with real API)
    api_url = "https://api.example.com/data"

```

```

# Step 1: Fetch data

df = await fetch_api_data(api_url)


if df.empty:

    logging.warning("No data fetched. Exiting pipeline.")

    return


# Step 2: Clean and transform data

df_clean = clean_transform_data(df)


# Step 3: Run ML model if target column exists

if 'target' in df_clean.columns:

    run_ml_model(df_clean, target_col='target')

else:

    logging.warning("'target' column missing. Skipping ML model.")


# =====

# Run the Async Pipeline

# =====

if __name__ == "__main__":

    asyncio.run(main())

    logging.info("Data pipeline execution completed successfully.")

```